

Sprint Backlog - Sprint 1

Projeto: Monitoria App

Sprint: 1 - Perfis e Autenticação

Período: 07/05/2026 a 13/05/2026

QM: Willian Gomes Pessoa

Time: Bruna, Thais, João Pedro Bianco, Willian Gomes Pessoa, Pedro Chaves, Gabriel dos Reis Benevides, Gustavo Blandy de Oliveira

Meta da Sprint

Implementar o sistema de autenticação e gestão de perfis, habilitando login seguro, controle de acesso por papel e cadastro de usuários com fluxo de primeiro acesso.

Itens Comprometidos

O Sprint 1 comprometeu 11 itens: 8 Must, 2 Should e 1 Could. Os itens de infraestrutura (EP00) foram pré-requisito para as histórias de usuário (EP01).

Must - EP00 (Infraestrutura): TT01 (Definir stack), TT02 (Modelar banco de dados), TT03 (Configurar ambiente local), TT04 (Criar skeleton do projeto), TT05 (Conexão com MySQL), TT06 (Deploy na nuvem). Responsáveis: Willian (TT01, TT06) e João Pedro (TT02, TT03, TT04, TT05). Todos concluídos.

Must - EP01 (Autenticação): US01 (Admin cadastra usuários) e US02 (Login com email e senha). Responsáveis: João Pedro e Willian em conjunto. Ambos concluídos.

Should - EP01: US03 (Monitor edita perfil) e US04 (Admin desativa usuário). Responsável: Willian. Ambos concluídos.

Could - EP01: US05 (Admin reseta senha). Won't do - postergado. O escopo de Must e Should estava comprometido e a criação de usuário (US01) já cobre o caso de uso mais crítico com senha temporária exibida ao admin.

Detalhamento dos Itens

TT01 - Definir e documentar a stack

Critérios de aceitação:

- Framework Python escolhido e documentado
- Versão do Python definida e registrada
- Versão do MySQL definida e registrada
- Documento disponível no repositório em docs/

Resultado: Stack definida como Python 3.12, Flask 3.0.3, MySQL 8.0. Documentado em ``docs/stack.md``.

TT02 - Modelar banco de dados

Critérios de aceitação:

- Diagrama ER criado com as entidades principais
- Script SQL de criação do schema disponível no repositório
- Relacionamentos e chaves estrangeiras definidos
- Tipos de dados e constraints explicitados

Resultado: Schema SQL com 4 tabelas criadas: ``usuarios``, ``password_reset_tokens``, ``disciplinas``, ``monitorias``. Diagrama ER em ``docs/modelagem-banco.md``.

TT03 - Configurar ambiente de desenvolvimento local

Critérios de aceitação:

- Guia de setup documentado em docs/setup.md
- Cobrir instalação do Python, MySQL, dependências e variáveis de ambiente
- Qualquer membro do time consegue rodar o projeto do zero sem ajuda verbal

Resultado: Guia publicado com Docker Compose para MySQL, venv Python e script ``create_admin.py`` para criação do primeiro admin.

TT04 - Criar estrutura base do projeto (skeleton)

Critérios de aceitação:

- Estrutura de pastas definida e criada
- Servidor sobe sem erros e responde a GET /health com HTTP 200
- Dependências listadas em requirements.txt com versões fixadas
- .gitignore configurado para Python e variáveis de ambiente

Resultado: App factory com ``create_app()`` e blueprints por domínio: ``auth``, ``usuarios``, ``disciplinas``, ``agenda``, ``registros``, ``relatorios``. Templates em ``frontend/`` separados do backend.

TT05 - Configurar conexão da aplicação com MySQL

Critérios de aceitação:

- Aplicação conecta ao banco na inicialização
- Configuração de banco via variável de ambiente (não hardcoded)
- Erro de conexão logado de forma clara no console

Resultado: Connection pool de 5 conexões reutilizáveis via ``MySQLConnectionPool``.
Configuração lida de variáveis de ambiente pela classe ``Config``.

TT06 - Publicar aplicação em servidor na nuvem

Critérios de aceitação:

- Aplicação acessível publicamente via URL
- Deploy automatizado a cada push na branch principal
- Schema aplicado automaticamente no primeiro boot

Resultado: Deploy configurado no Railway com ``Procfile`` (gunicorn) e ``railpack.json``. Schema auto-aplicado via ``connection.py``. URL: <https://web-production-1f724.up.railway.app>

US01 - Admin cadastra usuários com perfis

Cenário 1: Cadastro bem-sucedido

...

Given: admin autenticado na tela de gestão de usuários

When: preenche nome, email e papel (aluno/monitor/professor/admin) e confirma

Then: usuário é criado com status "Pendente" e senha temporária exibida ao admin

...

Cenário 2: Email duplicado

...

Given: admin tenta cadastrar email já existente no sistema

When: confirma o cadastro

Then: sistema rejeita com mensagem "Email já cadastrado"

...

Cenário 3: Campos obrigatórios ausentes

...

Given: admin está na tela de cadastro

When: tenta confirmar sem preencher nome, email ou papel

Then: sistema impede o envio e aponta o campo faltando

...

Resultado: Implementado em `usuarios/`. Senha temporária gerada com `secrets.choice` (10 caracteres alfanuméricos). Status inicial PENDENTE. Reset e desativação disponíveis na mesma tela.

US02 - Usuário faz login com email e senha

Cenário 1: Login bem-sucedido

...

Given: usuário com status "Ativo" na tela de login

When: informa email e senha corretos

Then: é autenticado e redirecionado conforme seu papel

...

Cenário 2: Credenciais inválidas

...

Given: usuário na tela de login

When: informa email ou senha incorretos

Then: sistema exibe mensagem genérica sem revelar qual campo está errado

...

Cenário 3: Primeiro acesso

...

Given: usuário com status "Pendente" na tela de login

When: informa email e senha temporária corretos

Then: é redirecionado para troca de senha obrigatória

...

Cenário 4: Usuário inativo

...

Given: usuário com status "Inativo"

When: informa credenciais corretas

Then: login é negado com mensagem de conta inativa

...

Resultado: Implementado em `auth/`. Email normalizado para lowercase. Sessão por cookie com HTTPOnly. Redirecionamento pós-login por papel.

US03 - Monitor edita seu perfil

Cenário 1: Atualização bem-sucedida

...

Given: monitor autenticado na tela de perfil

When: atualiza contato ou disponibilidade e confirma

Then: dados são salvos e exibidos atualizados

...

Resultado: Implementado em `GET/POST /usuarios/my-profile`. Campos `contato` e `disponibilidade` editáveis. Acesso restrito por `@login\_required`.

US04 - Admin desativa um usuário

Cenário 1: Desativação bem-sucedida

...

Given: admin na tela de gestão de usuários

When: clica em "Desativar" para um usuário ativo

Then: usuário tem status alterado para Inativo e não consegue mais fazer login

...

Cenário 2: Admin tenta desativar a si mesmo

...

Given: admin na tela de gestão

When: tenta desativar sua própria conta

Then: sistema rejeita com mensagem de erro (proteção contra lock-out)

...

Resultado: Implementado em `POST /usuarios/<id>/desativar`. Validação anti-lock-out no service. Bloqueio de login verificado em `authenticate\_user()`.

US05 - Admin reseta a senha de um usuário

Decisão: Item classificado como Could no MoSCoW. O time optou por não implementar nesta sprint - o escopo de Must e Should já estava comprometido. A funcionalidade de reset de senha temporária está parcialmente coberta pelo fluxo de criação de usuário (US01), que gera senha temporária e exhibe ao admin.

Status: Won't do - postergado para Sprint futura.

Resumo da Sprint

O Sprint 1 entregou 10 dos 11 itens comprometidos. Todos os 8 itens Must e os 2 itens Should foram concluídos. O único item não entregue foi US05 (Could), decisão deliberada do time dado o escopo já comprometido. Velocidade da sprint: 10/11 itens (91%).

Definition of Done - Verificação

O DoD do projeto exige: código escrito e critérios de aceitação atendidos; sem flags pendentes do QM; dev explicou a lógica técnica ao QM; QM atualizou o Sprint Tales. Todos os 10 itens entregues foram verificados contra esses critérios.

TT01 - Definir e documentar a stack

Stack definida (Python 3.12, Flask 3.0.3, MySQL 8.0) e documentada em `docs/stack.md`. Sem flags do QM. Lógica técnica explicada ao QM: escolha por Flask pela simplicidade e curva de aprendizado; MySQL pelo modelo relacional; queries SQL diretas para transparência na revisão. Sprint Tales atualizado.

TT02 - Modelar banco de dados

Schema SQL com 4 tabelas (`usuarios`, `password\_reset\_tokens`, `disciplinas`, `monitorias`) disponível em `backend/db/schema.sql`. FKs, constraints UNIQUE e ON DELETE CASCADE declarados. Diagrama ER em `docs/modelagem-banco.md`. Sem flags do QM. Sprint Tales atualizado.

TT03 - Configurar ambiente de desenvolvimento local

Guia de setup publicado em `docs/setup.md`. Cobre Docker Compose para MySQL, criação de env, variáveis de ambiente e script de criação do admin. Testado em mais de um ambiente. Sem flags do QM. Sprint Tales atualizado.

TT04 - Criar estrutura base do projeto (skeleton)

App factory com `create\_app()` implementado. Rota `GET /health` retorna HTTP 200. Blueprints registrados para todos os domínios. Templates e estáticos em `frontend/` separados do backend. Sem flags do QM. Sprint Tales atualizado.

TT05 - Configurar conexão da aplicação com MySQL

Connection pool de 5 conexões configurado. Nenhuma credencial hardcoded - tudo via variáveis de ambiente. Falha na inicialização logada com `logging.exception`. Sem flags do QM. Sprint Tales atualizado.

TT06 - Publicar aplicação em servidor na nuvem

Aplicação acessível em <https://web-production-1f724.up.railway.app>. Deploy ativo no Railway

com `Procfile` (gunicorn) e `railpack.json`. Schema auto-aplicado via `connection.py` a cada boot. `config.py` adaptado para aceitar variáveis de ambiente no formato Railway. Sem flags do QM. Sprint Tales atualizado.

US01 - Admin cadastra usuários com perfis

Todos os 3 cenários BDD verificados: cadastro bem-sucedido, rejeição de email duplicado, bloqueio de campos obrigatórios ausentes. Senha temporária gerada com `secrets.choice` e exibida ao admin. Sem flags do QM. Lógica técnica explicada ao QM. Sprint Tales atualizado.

US02 - Usuário faz login com email e senha

Todos os 4 cenários BDD verificados: login bem-sucedido com redirecionamento por papel; erro genérico para credenciais inválidas; redirecionamento para primeiro acesso com status PENDENTE; bloqueio de login com status INATIVO. Email normalizado para lowercase. Sessão por cookie com HTTPOnly. Sem flags do QM. Sprint Tales atualizado.

US03 - Monitor edita seu perfil

Cenário BDD verificado: atualização de `contato` e `disponibilidade` persistida no banco. Acesso controlado por `@login\_required` - usuário não autenticado é redirecionado ao login. Sem flags do QM. Sprint Tales atualizado.

US04 - Admin desativa um usuário

Ambos os cenários BDD verificados: desativação de outro usuário bem-sucedida; tentativa de auto-desativação rejeitada pelo service. Usuário inativo bloqueado no fluxo de login. Sem flags do QM. Sprint Tales atualizado.